

Delta Quant for Rotation Based Quantization

Ahmad Fraij

QuaRot and DuQuant

- QuaRot
 - Deals with outliers in activations by rotating the activations using randomized Hadamard transformations
 - Data-independent and calibration free method
- DuQuant:
 - Uses dual transformations: rotations and permutations
 - Local redistribution of feature outliers using block diagonal rotation matrices
 - Global redistribution of outliers across different blocks using zigzag permutation

Applying DeltaQuant to QuaRot and DuQuant

- Quantizing difference of two rotated matrices is the same as quantizing the rotation of the difference because rotation by an orthogonal matrix is a linear map that distributes over subtraction

$$\Delta_t = X_t - X_{t-1}.$$

$$RX_t - RX_{t-1} = R(X_t - X_{t-1}) = R\Delta_t,$$

$$\mathcal{Q}(RX_t - RX_{t-1}) = \mathcal{Q}(R\Delta_t).$$

Applying DeltaQuant to QuaRot

- Added quantization for anchor bits to make the comparison with the baseline approach fair.
- Applied rotations to the activations before quantizing the difference between rotated activations

```
def forward(self, x):
    x_dtype = x.dtype
    x_had = self._apply_hadamard(x)

    if self._x_had_cache is None:
        if self.anchor_bits < 16:
            x_had_q = _fake_quant(x_had.float(), self.anchor_bits, self.clip_ratio).to(x_dtype)
        else:
            x_had_q = x_had
        out = self.module(x_had_q)
        self._x_had_cache = x_had.detach().clone()
        self._output_cache = out.detach().clone()
    else:
        delta = x_had - self._x_had_cache
        delta_q = _fake_quant(delta.float(), self.bits, self.clip_ratio).to(x_dtype)
        delta_out = self.module(delta_q)
        out = self._output_cache + delta_out
        self._x_had_cache = x_had.detach().clone()
        self._output_cache = out.detach().clone()

    return out
```

Applying DeltaQuant to DuQuant

- Similar to QuaRot approach but applied an inverse rotation matrix after quantization because weights are kept in FP16.

```
def forward(self, x):
    x_rot = self._rotate_forward(x)
    if self._x_rot_cache is None:
        x_in = _fake_quant(x_rot.float(), self.anchor_bits, self.clip_ratio).to(x.dtype) if self.anchor_bits < 16 else x_rot
        x_in = self._rotate_inverse(x_in)
        out = self.module(x_in)
        self._x_rot_cache = x_rot.detach().clone()
        self._output_cache = out.detach().clone()
    else:
        delta_rot = x_rot - self._x_rot_cache
        delta_rot_q = _fake_quant(delta_rot, self.bits, self.clip_ratio)
        delta_q = self._rotate_inverse(delta_rot_q)
        delta_out = self.module(delta_q)
        out = self._output_cache + delta_out
        self._x_rot_cache = x_rot.detach().clone()
        self._output_cache = out.detach().clone()
    return out
```

DeltaQuaRot Evaluation Results

- Delta Quant degrades the results more than baseline method for QuaRot
- Quantizing the anchor improves the results in the W4A8 case

LLaDA-8B-Base accuracy under QuaRot rotation quantization setting. $\Delta\%$ is the relative change of the group average w.r.t. the FP16 base.

Model	Setting	Method	Math				Code			
			GSM8K	MATH	Avg	$\Delta\%$	HumanEval	MBPP	Avg	$\Delta\%$
LLaDA-8B-Base	FP16	Base	70.36	20.43	45.40	–	32.32	40.40	36.36	–
	W4A8	QuaRot	52.77	12.43	32.60	↓28.19%	22.56	32.60	27.58	↓24.1%
		DeltaQuaRot	51.93	11.14	31.54	↓30.53%	21.95	31.40	26.68	↓26.60%
		DeltaQuaRot (Quantized Anchor)	51.93	11.85	31.89	↓29.76%	22.56	31.80	27.18	↓25.25%
	W4A4	QuaRot	37.68	9.57	23.63	↓47.95%	17.68	26.6	22.14	↓39.10%
		DeltaQuaRot	33.43	5.29	19.36	↓57.40%	14.63	22.60	18.62	↓48.79%
		DeltaQuaRot (Quantized Anchor)	25.93	3.43	14.68	↓67.67%	14.63	18.60	16.62	↓54.29%

DeltaDuQuant Evaluation Results

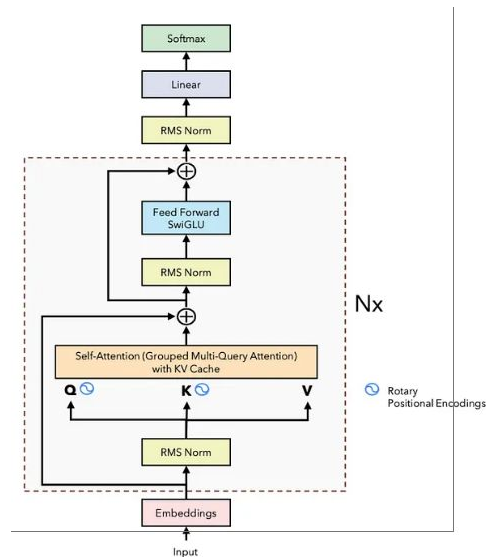
- DeltaQuant method improves the results significantly in the DuQuant case
- DeltaQuant fails completely in W4A4 case
- Quantizing the anchors also improves the results in the coding benchmarks

LLaDA-8B-Base accuracy under DuQuant rotation quantization setting. $\Delta\%$ is the relative change of the group average w.r.t. the FP16 base.

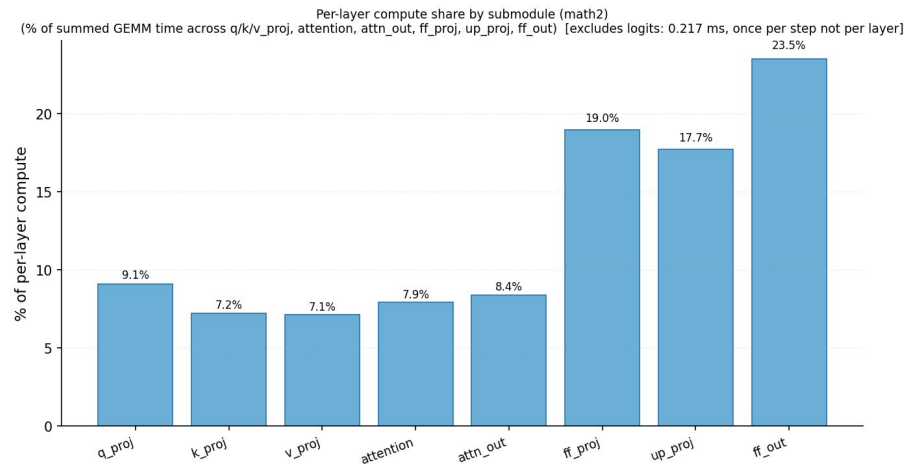
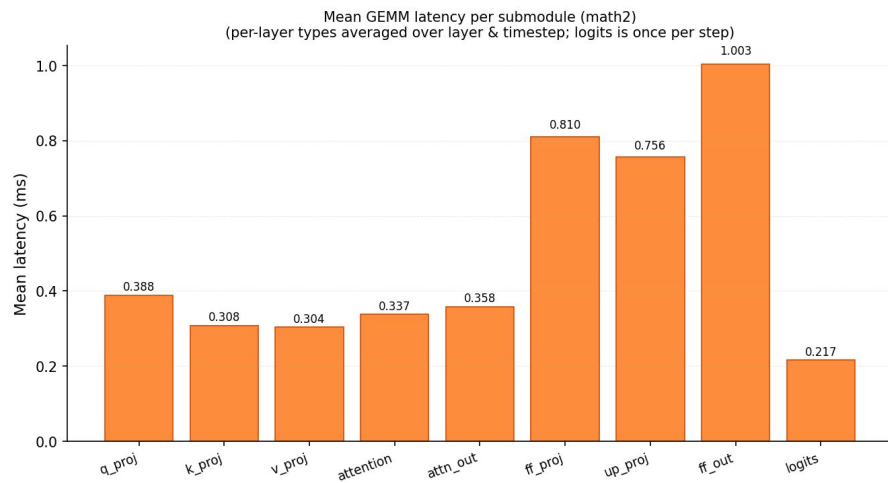
Model	Setting	Method	Math				Code			
			GSM8K	MATH	Avg	$\Delta\%$	HumanEval	MBPP	Avg	$\Delta\%$
LLaDA-8B-Base	FP16	Base	70.36	20.43	45.40	–	32.32	40.40	36.36	–
	W4A8	DuQuant	–	15.29	–	↓–%	29.88	35.20	32.54	↓10.50%
	W16A8	DeltaDuQuant	70.28	21.43	45.86	↑1.00%	32.93	39.80	36.37	↑0.03%
		DeltaDuQuant (Quantized Anchor)	68.99	–	–	↓–%	32.93	40.20	36.57	↑0.57%
	W4A4	DuQuant	–	16.43	–	↓–%	24.39	32.00	28.20	↓22.44%
	W16A4	DeltaDuQuant	0	–	–	↓–%	0	0	–	↓–%
		DeltaDuQuant (Quantized Anchor)	0	–	–	↓–%	0	0	–	↓–%

Llada Architecture and Submodules

- There are 7 linear projections per block but only 4 unique inputs:
 - qkv_in: q_proj, k_proj and v_proj share same input (attn_norm output)
 - attn_out_in: attn_out receives result of attention after concatenating heads
 - ff_in: ff_proj and up_proj both share same input (ff_norm output)
 - ff_out_in: $\text{SiLU}(\text{ff_proj}) \odot \text{up_proj}$



GEMM Computation time for submodules



Plan for Next Week

- Evaluate the W4A8 and W4A4 settings for DeltaDuQuant
- Build DeltaQuant approach that involves periodic re-anchoring
- Analyze how to choose the re-anchoring points in a way that balances the tradeoff between compute speed and inference accuracy